

LAPORAN TUGAS BESAR

IF25-21013 STRATEGI ALGORITMA

PEMANFAATAN ALGORITMA GREEDY DALAM PEMBUATAN BOT PERMAINAN ROBOCODE TANK ROYALE



Disusun oleh:

Kezia Adelina Tamba	(124140046)
Nathania Calista Hutapea	(124140101)
Sahal Alvin Zairy	(124140167)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
SEMESTER GENAP 2026/2027**

DAFTAR ISI

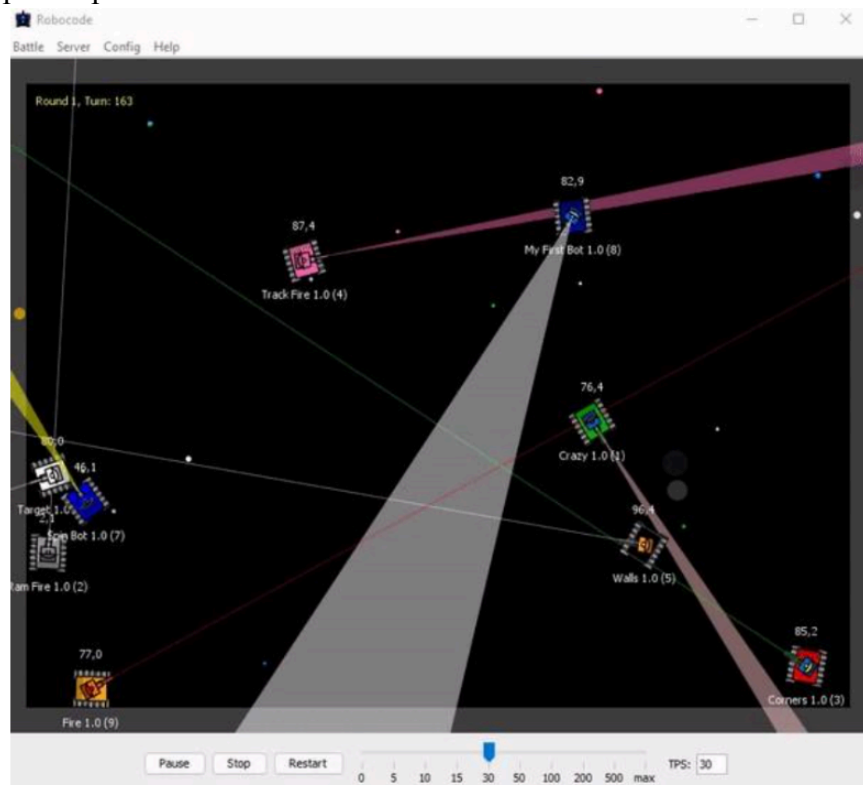
1. BAB 1 DESKRIPSI TUGAS.....	4
1.1. Latar Belakang.....	4
Gambar 1. Robocode Tank Royale.....	4
1.2. Komponen Permainan.....	5
1.2.1. Rounds dan Turns.....	5
1.2.2. Energi dan Peluru.....	5
1.2.3. Gun Heat.....	5
1.2.4. Pergerakan dan Turning.....	5
1.2.5. Pemindaian Radar.....	5
1.2.6. Sistem Skor.....	6
1.3. Rumusan Masalah.....	6
1.4. Tujuan.....	6
1.5. Batasan Masalah.....	7
2. BAB 2 LANDASAN TEORI.....	8
2.1. Algoritma Greedy.....	8
2.2. Karakteristik dan Batasan Greedy.....	8
2.3. Fungsi Objektif dan Heuristik.....	8
2.4. Local Optimum vs Global Optimum.....	9
2.5. Implementasi Bot dalam Robocode Tank Royale (C# .NET).....	9
3. BAB 3 APLIKASI STRATEGI GREEDY.....	11
3.1. Mapping Persoalan Robocode ke Elemen Algoritma Greedy.....	11
3.1.1. Himpunan Kandidat.....	11
3.1.2. Himpunan Solusi.....	11
3.1.3. Fungsi Solusi.....	11
3.1.4. Fungsi Seleksi.....	11
3.1.5. Fungsi Kelayakan.....	11
3.1.6. Fungsi Objektif.....	11
3.2. Eksplorasi Empat Strategi Greedy.....	12
3.2.1. Bot Utama: SixSeven (Greedy Local Optimization with Adaptive Movement).....	12
Gambar 5. Six Seven memilih menembak bot yang berjarak kecil.....	12
3.2.2. Alternatif 1: Keyju (Greedy Berbasis Energi Terendah).....	15
Gambar 6. Keyju memilih bot energi rendah.....	15
3.2.3. Alternatif 2: 67alter (Greedy Berbasis Zona Aman dan Zigzag).....	17
Gambar 7. alter67 bertahan dengan survival mode.....	18
3.2.4. Alternatif 3: TOP67 (Greedy Berbasis Confidence Scoring).....	20
Gambar 8. TOP67 bergerak dengan random.....	21
3.3. Perbandingan Keempat Strategi.....	23
3.4. Alasan Pemilihan Strategi Utama.....	24
4. BAB 4. IMPLEMENTASI DAN PENGUJIAN.....	26
4.1. Pseudocode Keempat Strategi.....	26
4.1.1. SixSeven, Greedy Local Optimization with Adaptive Movement.....	26
4.1.2. Keyju, Orbit-Based Greedy with Adaptive Firepower.....	26

4.1.3. 67alter, Greedy Berbasis Zona Aman dan Zigzag.....	28
4.1.4. TOP67, Greedy Berbasis Confidence Scoring.....	28
4.2. Struktur Data dan Fungsi Penting.....	29
4.3. Pengujian.....	32
4.3.1. Dokumentasi Hasil Pengujian.....	32
Gambar 2. Percobaan Pertama.....	32
Gambar 3. Percobaan Kedua.....	33
Gambar 4. Percobaan Ketiga.....	33
4.3.2. Analisis Hasil Pengujian.....	33
4.3.3. Analisis Keberhasilan Greedy.....	34
5. BAB 5. KESIMPULAN DAN SARAN.....	37
5.1. Kesimpulan.....	37
5.2. Efektivitas Greedy pada Robocode.....	37
5.3. Keterbatasan.....	37
5.4. Potensi Pengembangan.....	38
LAMPIRAN.....	39
Tautan Repository GitHub.....	39
Tautan Video YouTube.....	39
Checklist Spesifikasi.....	39
DAFTAR PUSTAKA.....	40

1. BAB 1 DESKRIPSI TUGAS

1.1. Latar Belakang

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, IF25--21013 Strategi Algoritma – Tugas Besar 1 karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari versi asli/pertama permainan ini. Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.



Gambar 1. Robocode Tank Royale

Pada Tugas Besar IF25-21013 Strategi Algoritma semester genap tahun 2026/2027 berfokus pada penerapan algoritma greedy dalam pembuatan bot untuk permainan Robocode Tank Royale. Setiap kelompok diwajibkan membuat empat buah bot menggunakan bahasa pemrograman

C# dengan platform .NET, di mana satu bot ditetapkan sebagai bot utama yang akan dikompetisikan, dan tiga bot lainnya berfungsi sebagai alternatif dengan strategi berbeda. Masing-masing bot harus mengimplementasikan strategi greedy yang unik dengan heuristik yang berbeda-beda.

Tujuan akhir dari tugas besar ini bukan sekadar membuat bot yang bisa bergerak dan menembak, melainkan merancang bot yang secara konsisten mampu memaksimalkan skor akhir pertempuran melalui pengambilan keputusan lokal yang optimal pada setiap turn. Ini adalah tantangan nyata dalam desain algoritma, karena lingkungan Robocode bersifat dinamis, kompetitif, dan penuh ketidakpastia

1.2. Komponen Permainan

1.2.1. Rounds dan Turns

Satuan waktu terkecil dalam Robocode adalah turn. Pada setiap turn, bot menerima informasi terbaru mengenai posisi dan orientasinya, kemudian memutuskan aksi apa yang akan diambil. Keputusan tersebut meliputi arah pergerakan, rotasi gun, rotasi radar, dan apakah akan menembak atau tidak. Setiap keputusan harus diselesaikan dalam batas waktu turn timeout, yaitu antara 30-50 ms.

1.2.2. Energi dan Peluru

Semua bot memulai permainan dengan 100 poin energi. Bot akan kehilangan energi jika ditembak atau ditabrak, dan juga kehilangan energi setiap kali menembakkan meriam. Sebaliknya, bot mendapatkan energi jika peluru dari meriamnya mengenai musuh, yaitu sebesar 3 kali lipat dari energi yang digunakan untuk menembakkan peluru tersebut. Semakin besar daya tembak (firepower), semakin berat peluru dan semakin lambat gerakannya, tetapi semakin besar damage yang dihasilkan.

1.2.3. Gun Heat

Setiap kali bot menembak, meriam mengalami pemanasan (gun heat). Bot tidak bisa menembak kembali sebelum gun heat turun ke nol. Peluru dengan daya tinggi menghasilkan panas lebih besar dibanding peluru ringan. Di awal setiap ronde, meriam juga sudah dalam kondisi panas dan perlu waktu untuk mendingin sebelum bisa digunakan pertama kalinya.

1.2.4. Pergerakan dan Turning

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Percepatan maksimum adalah 1 unit per turn, dengan pengereman maksimum 2 unit per turn. Setiap bagian tubuh bot memiliki kecepatan rotasi berbeda: radar dapat berputar hingga 45 derajat per turn, gun hingga 20 derajat per turn, dan body hingga 10 derajat per turn (bergantung pada kecepatan bot saat itu). Semakin cepat bot bergerak, semakin terbatas kemampuannya untuk berbelok.

1.2.5. Pemindaian Radar

Radar mendeteksi musuh dalam jangkauan hingga 1200 piksel. Penting untuk dipahami bahwa bot hanya dapat mendeteksi musuh yang berada dalam scan arc, yaitu sudut sapuan radar dari posisi sebelumnya ke posisi saat ini dalam satu turn. Jika radar tidak bergerak sama sekali dalam suatu turn, scan arc menjadi nol dan tidak ada musuh yang dapat terdeteksi. Oleh karena itu, radar harus selalu diputar agar bot dapat memantau kondisi arena.

1.2.6. Sistem Skor

Skor akhir bot merupakan akumulasi dari enam komponen berikut:

- Bullet Damage: poin sebesar damage yang ditimbulkan kepada musuh melalui peluru.
- Bullet Damage Bonus: 20% dari total damage ke musuh yang berhasil dibunuh.
- Survival Score: 50 poin setiap kali ada bot lain yang mati pada ronde yang sama.
- Last Survival Bonus: 10 poin dikalikan jumlah musuh bagi bot terakhir yang bertahan.
- Ram Damage: 2 kali lipat damage yang ditimbulkan melalui tabrakan.
- Ram Damage Bonus: 30% dari total damage ke musuh yang terbunuh akibat tabrakan.

1.3. Rumusan Masalah

1. Bagaimana representasi permasalahan dalam Robocode Tank Royale sebagai sebuah persoalan yang dapat diselesaikan dengan pendekatan algoritma greedy?
2. Bagaimana pemanfaatan nilai heuristik dalam menentukan aksi terbaik secara lokal pada aspek pergerakan, pembidikan, pemindaian radar, dan pelepasan tembakan?
3. Apa perbedaan mendasar dari fungsi tujuan (*objective function*) yang digunakan pada keempat bot?
4. Bagaimana rancangan struktur data, fungsi-fungsi kunci, serta representasi *pseudocode* yang diimplementasikan pada masing-masing bot?
5. Bot manakah yang menunjukkan tingkat efektivitas tertinggi berdasarkan analisis perbandingan strategi, kualitas implementasi, serta hasil pengujian fungsional?

1.4. Tujuan

1. Memodelkan permasalahan dalam Robocode Tank Royale ke dalam bentuk yang dapat dipecahkan menggunakan algoritma greedy.
2. Menerapkan nilai heuristik untuk memilih aksi lokal yang optimal pada aspek pergerakan bot, pembidikan ke arah musuh, pemindaian radar, serta pelepasan tembakan.
3. Menganalisis dan membandingkan perbedaan fungsi objektif (*objective function*) yang digunakan oleh keempat bot.
4. Mengimplementasikan struktur data, fungsi-fungsi penting, serta menyusun *pseudocode* dari setiap bot yang dibuat.

5. Mengevaluasi dan menentukan bot paling efektif berdasarkan analisis strategi, kualitas implementasi, serta hasil pengujian fungsional yang telah dilakukan.

1.5. Batasan Masalah

Penulisan laporan ini memusatkan perhatian pada penerapan algoritma greedy dalam bot Robocode Tank Royale yang dikembangkan menggunakan C# dan API Robocode Tank Royale. Kajian meliputi strategi bergerak, strategi membidik, pemindaian radar, nilai heuristik, fungsi tujuan, minimalisasi risiko, maksimalisasi skor, serta pengambilan keputusan waktu nyata. Sementara itu, laporan ini tidak membahas perubahan pada *engine* permainan, rancangan antarmuka pengguna, atau implementasi *machine learning* tingkat kompleks, kecuali sebagai rekomendasi untuk pengembangan lebih lanjut.

2. BAB 2 LANDASAN TEORI

2.1. Algoritma Greedy

Algoritma greedy adalah pendekatan pemecahan masalah yang membangun solusi secara bertahap dengan selalu memilih opsi terbaik yang tersedia pada setiap langkah, tanpa mempertimbangkan konsekuensi jangka panjang dari pilihan tersebut. Kata "greedy" (rakus) menggambarkan perilaku algoritma yang selalu mengambil keuntungan maksimum dari setiap langkah lokal.

Secara formal, algoritma greedy bekerja melalui lima elemen utama: himpunan kandidat (semua kemungkinan pilihan), himpunan solusi (kombinasi pilihan yang sudah diambil), fungsi seleksi (cara memilih kandidat terbaik), fungsi kelayakan (menguji apakah pilihan valid), dan fungsi objektif (ukuran kualitas solusi).

2.2. Karakteristik dan Batasan Greedy

Algoritma greedy memiliki karakteristik utama yaitu mengambil keputusan optimal lokal pada setiap langkah (local optimum) dengan harapan menghasilkan solusi optimal global (global optimum), tanpa melakukan pelacakan mundur (backtracking) atau evaluasi jangka panjang. Pada implementasi keempat bot—SixSeven, Keyju, 67alter, dan TOP67—karakteristik ini terlihat pada pemilihan target terdekat atau terlemah, penentuan kekuatan tembakan berdasarkan jarak sesaat, serta pemilihan titik pergerakan dengan nilai risiko terendah. Namun, algoritma greedy juga memiliki batasan fundamental, yaitu tidak menjamin solusi optimal global untuk permasalahan yang bersifat non-linear atau memiliki ketergantungan antar keputusan. Dalam konteks Robocode, batasan ini muncul ketika bot terlalu fokus pada satu musuh (target terdekat) sehingga mengabaikan ancaman dari musuh lain, atau ketika bot memilih pergerakan aman jangka pendek namun terperangkap di sudut arena.

Selain itu, greedy juga rentan terhadap overfitting terhadap kondisi saat ini, misalnya bot yang menembak dengan power maksimal karena jarak dekat tanpa menyisakan energi untuk bertahan di turn berikutnya. Meski demikian, keterbatasan waktu turn timeout (30–50 ms) membuat pendekatan greedy tetap menjadi pilihan rasional karena kemampuannya mengambil keputusan cepat tanpa komputasi berat yang dibutuhkan oleh algoritma pencarian menyeluruh.

2.3. Fungsi Objektif dan Heuristik

Heuristic function merupakan fungsi penilaian cepat yang digunakan untuk memilih keputusan lokal terbaik tanpa harus

mengevaluasi seluruh kemungkinan solusi secara eksplisit. Pada Robocode Tank Royale, fungsi heuristik membantu bot mengambil keputusan real-time seperti menentukan arah gerakan teraman, memilih target paling mengancam, atau menentukan kekuatan tembakan optimal. Keempat bot yang dikembangkan—SixSeven, Keyju, 67alter, dan TOP67—masing-masing mengimplementasikan fungsi heuristik dengan karakteristik berbeda. SixSeven menggunakan `CalculateDangerLevel()` yang menilai risiko suatu titik berdasarkan jarak ke musuh dan faktor energi, serta `SelectNearestEnemy()` untuk memilih target terdekat. Keyju memprioritaskan musuh berenergi terendah sebagai target utama, dengan jarak sebagai tiebreaker, serta menyesuaikan kecepatan orbit berdasarkan rasio jarak terhadap jarak preferensi. 67alter menerapkan heuristik sederhana pada fire power berbasis jarak dan energi, serta pada zone control untuk tetap berada di tengah arena. Sementara TOP67 memiliki fungsi heuristik paling kompleks melalui `calculateFirePower()` bertahap berdasarkan jarak dan `aimConfidence` yang mempertimbangkan jarak, kecepatan musuh, serta sudut putaran meriam untuk menentukan waktu tembak yang tepat.

Secara umum, keempat bot ini membuktikan bahwa fungsi heuristik yang dirancang sesuai dengan karakter bot dapat meningkatkan efektivitas tempur secara signifikan. SixSeven lebih menekankan pada minimalisasi risiko pergerakan, Keyju pada efisiensi energi dan orbit konsisten, 67alter pada kesederhanaan dan prediktabilitas rendah, sedangkan TOP67 pada akurasi tembakan melalui confidence-based targeting. Meskipun pendekatan yang digunakan berbeda-beda, semuanya mengikuti prinsip greedy: memilih opsi dengan nilai heuristik terbaik pada setiap turn tanpa mempertimbangkan konsekuensi jangka panjang, sehingga cocok untuk lingkungan pertempuran yang dinamis dan membutuhkan keputusan cepat dalam batasan waktu turn timeout.

2.4. Local Optimum vs Global Optimum

Local optimum adalah solusi terbaik pada suatu langkah berdasarkan informasi yang tersedia saat itu, sedangkan *global optimum* adalah solusi terbaik secara keseluruhan. Algoritma greedy bekerja dengan selalu memilih *local optimum* pada setiap langkah, berharap akumulasinya menghasilkan *global optimum*. Namun, hal ini tidak selalu terjamin. Pada keempat bot, pemilihan target terdekat (SixSeven), musuh berenergi terendah (Keyju), atau *fire power* berdasarkan jarak (67alter dan TOP67) merupakan contoh *local optimum*. Kelemahannya muncul ketika pilihan lokal terbaik justru mengarah pada hasil akhir yang kurang optimal, misalnya bot yang fokus mengejar satu musuh tetapi terkena tembakan dari musuh lain.

2.5. Implementasi Bot dalam Robocode Tank Royale (C# .NET)

Keempat bot dalam tugas besar ini diimplementasikan menggunakan bahasa C# dengan target framework .NET 6.0 hingga .NET 10.0, serta memanfaatkan *library* Robocode.TankRoyale.BotApi versi 0.30.0 untuk berinteraksi dengan *game engine*. Masing-masing bot

didefinisikan sebagai kelas yang mewarisi kelas dasar Bot dari API tersebut, kemudian mengimplementasikan metode Run() sebagai *main loop* serta berbagai *event handler* seperti OnScannedBot(), OnHitByBullet(), OnHitBot(), dan OnHitWall(). Struktur proyek setiap bot dilengkapi dengan file konfigurasi .json yang berisi metadata bot (nama, versi, penulis, platform), serta file .cmd atau .sh untuk menjalankan bot dalam mode *development* (rebuild otomatis) maupun *release* (tanpa rebuild). Bot SixSeven (bot utama) mengimplementasikan strategi *stop-and-go movement* untuk mode 1v1, *linear targeting* untuk prediksi posisi musuh, serta fungsi risiko (CalculateDangerLevel()) untuk menentukan titik pergerakan teraman. Bot Keyju fokus pada *orbit movement* dengan jarak preferensi 250 piksel, pemilihan target berdasarkan energi terendah, serta mekanisme *ramming* untuk musuh yang energinya tinggal sedikit. Bot 67alter mengusung pendekatan *zigzag dinamis* di tengah arena dengan periode acak, serta *zone control* untuk selalu berada di zona aman (30-70% lebar/tinggi arena). Sementara bot TOP67 mengimplementasikan mode defensif saat energi di bawah 30, *wavy movement* di sepanjang sisi arena, serta sistem *aim confidence* yang mempertimbangkan jarak, kecepatan musuh, dan sudut putaran meriam sebelum memutuskan menembak.

Dari segi teknis, keempat bot memanfaatkan fitur API seperti SetTurnRadarLeft/Right() untuk pergerakan radar, SetTurnGunLeft/Right() untuk membidik, SetForward/Back() untuk pergerakan, serta SetFire() untuk menembak. Bot SixSeven dan Keyju menggunakan AdjustGunForBodyTurn dan AdjustRadarForGunTurn untuk memungkinkan setiap komponen (body, gun, radar) berputar secara independen. Bot SixSeven juga mengimplementasikan kelas bantu PointD dan RectangleD untuk operasi geometri, serta metode GetShortestDistanceToSegment() untuk mendeteksi halangan jalur pergerakan. Bot Keyju menggunakan pendekatan *iterative prediction* sebanyak 100 iterasi untuk memprediksi posisi musuh, sementara bot TOP67 menggunakan *predictive targeting* berbasis waktu tempuh peluru. Seluruh bot juga menangani *event* tabrakan dengan bot lain (OnHitBot) dan dinding (OnHitWall) untuk beradaptasi dengan perubahan kondisi medan perang secara *real-time*. Implementasi ini menunjukkan bahwa algoritma greedy dapat diwujudkan dalam kode yang efisien dan responsif, dengan tetap mematuhi batasan *turn timeout* (30-50 ms) yang diberlakukan oleh *game engine*.

3. BAB 3 APLIKASI STRATEGI GREEDY

3.1. Mapping Persoalan Robocode ke Elemen Algoritma Greedy

Persoalan Robocode Tank Royale dapat dimodelkan sebagai persoalan pengambilan keputusan berurutan (*sequential decision making*) di mana pada setiap *turn*, bot harus memilih aksi terbaik dari sekian banyak kemungkinan. Berikut adalah pemetaan enam elemen algoritma greedy ke dalam konteks Robocode:

3.1.1. Himpunan Kandidat

Kumpulan aksi yang mungkin dilakukan bot pada suatu *turn*, misalnya: semua arah pergerakan ($0-360^\circ$), semua kekuatan tembakan ($0.1-3.0$), semua target musuh yang terdeteksi, atau semua titik koordinat di arena sebagai tujuan pergerakan.

3.1.2. Himpunan Solusi

Himpunan solusi adalah kombinasi aksi yang dipilih untuk *turn* saat ini: satu set perintah movement, gun, radar, dan firing yang akan dikirimkan ke server pada akhir *turn*. Dalam satu *turn*, bot bisa mengeksekusi pergerakan, rotasi, dan tembakan secara bersamaan.

3.1.3. Fungsi Solusi

Fungsi yang menentukan apakah rangkaian aksi yang diambil membawa bot menuju kemenangan (skor tertinggi), biasanya diukur dari akumulasi *bullet damage*, *survival score*, *ram damage*, dan bonus lainnya.

3.1.4. Fungsi Seleksi

Fungsi yang memilih kandidat terbaik dari himpunan kandidat berdasarkan nilai heuristik tertentu. Contoh: pilih target dengan jarak terdekat, atau pilih titik dengan nilai risiko terendah.

3.1.5. Fungsi Kelayakan

Fungsi yang memeriksa apakah suatu kandidat layak dipilih, misalnya: apakah energi cukup untuk menembak dengan power tertentu, apakah meriam tidak sedang overheat, atau apakah titik tujuan berada di dalam batas arena.

3.1.6. Fungsi Objektif

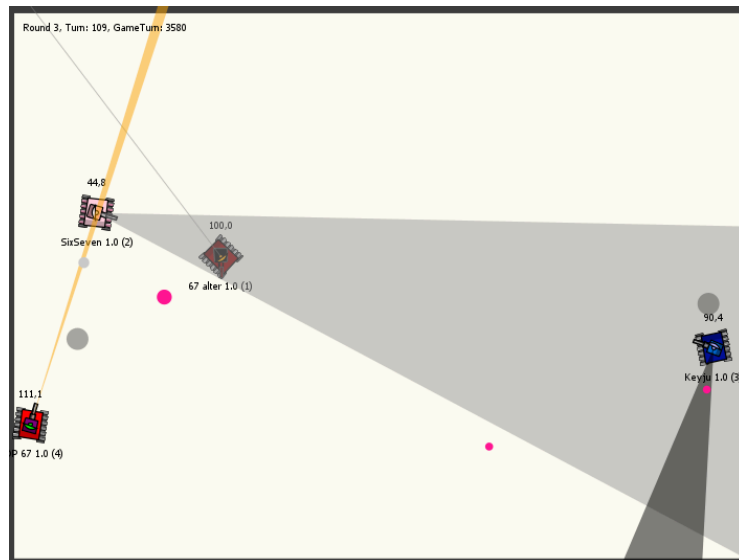
Fungsi yang ingin dimaksimalkan atau diminimalkan, seperti memaksimalkan *bullet damage*, memaksimalkan *survival score*, atau meminimalkan risiko terkena tembakan musuh.

3.2. Eksplorasi Empat Strategi Greedy

3.2.1. Bot Utama: SixSeven (Greedy Local Optimization with Adaptive Movement)

Ide Utama:

Bot SixSeven menggunakan pendekatan greedy local optimization di setiap komponen (gerak, tembak, radar) secara independen namun terkoordinasi. Bot selalu memilih opsi terbaik saat ini tanpa perencanaan jangka panjang: target terdekat untuk ditembak, titik tujuan dengan dangerScore terendah untuk dituju, dan musuh yang paling lama tidak discan untuk diprioritaskan radar. Rasionalnya adalah bahwa dalam pertempuran Robocode yang cepat, reaksi instan terhadap kondisi lokal lebih penting daripada strategi global yang kompleks.



Gambar 2. Six Seven memilih menembak bot yang berjarak kecil

Heuristik yang Digunakan:

Target Selection (Greedy): memilih musuh dengan jarak terkecil sebagai primary target karena jarak pendek meningkatkan akurasi tembakan dan damage output.

Movement Selection (Greedy): memilih titik tujuan dengan danger score terendah dari 360 kandidat titik di sekitar bot. Formula danger score: $0.08 / \text{jarak kuadrat ke posisi saat ini} + \text{Sigma}(\text{energyFactor dikali } (1 + \text{perpendicularFactor}) \text{ dibagi jarak kuadrat ke musuh})$. Semakin besar nilainya, semakin berbahaya.

Firepower Selection (Greedy): memilih bullet power terbesar yang memenuhi seluruh batasan: tidak melebihi 20 persen energi sendiri, tidak melebihi 30 persen energi musuh (menghindari overkill), proporsional terhadap jarak dengan rumus $0.65 + 150 \text{ dibagi jarak}$.

Radar Tracking (Greedy): memprioritaskan musuh dengan nilai lastSeen terkecil untuk di-scan ulang agar data musuh tidak terlalu kadaluarsa.

Stop-and-Go (khusus duel 1v1, jarak lebih dari 200): pola gerak periodik berupa 30 sampai 180 tick bergerak dan 5 sampai 10 tick diam untuk mengecoh musuh yang menggunakan linear targeting.

Mapping ke Elemen Greedy:

Himpunan Kandidat: untuk movement, 360 titik kandidat dengan radius dan sudut bervariasi di sekitar posisi saat ini; untuk targeting, semua musuh yang aktif di enemyRegistry; untuk firepower, nilai kontinu antara LOWEST_POWER (0.1) hingga HIGHEST_POWER (3.0).

Fungsi Seleksi: pilih titik dengan dangerScore minimum (movement); pilih musuh dengan jarak minimum (target); pilih firepower terbesar yang masih di bawah semua batasan (firepower).

Fungsi Kelayakan: titik tujuan harus berada dalam arenaBounds (margin 45 piksel) dan jalurnya tidak terhalang musuh (jarak segmen ke musuh lebih dari 22 piksel); tembakan hanya dilakukan jika GunHeat sama dengan 0 dan sudut gun dalam toleransi.

Fungsi Objektif: memaksimalkan bullet damage (melalui akurasi dan firepower optimal) dan survival score (melalui movement berbasis risiko).

Cara Pengambilan Keputusan:

Setup awal: radar dipisahkan dari gun dan body (AdjustRadarForGunTurn = true), memungkinkan radar berputar terus tanpa mengganggu bidikan.

Setiap turn: radar berputar kiri 60 derajat untuk scanning terus-menerus. Jika ada musuh yang lebih dari 20 turn tidak discan, radar diarahkan ke sana untuk memprioritaskan data yang fresh.

Saat scan musuh (OnScannedBot): simpan atau update data musuh (posisi, energi, kecepatan, arah). Jalankan SelectNearestEnemy untuk mencari musuh dengan jarak terkecil. Jika energi bot lebih dari 5 dan target valid: hitung bulletPower, hitung angleTolerance, panggil ExecuteLinearTargeting. Jika musuh diam, tembak langsung (ExecuteDirectTargeting). Jika musuh bergerak, selesaikan persamaan kuadrat untuk mencari impactTime, prediksi posisi musuh setelah waktu t, arahkan gun ke posisi prediksi, tembak jika GunHeat sama dengan 0 dan sudut dalam toleransi.

Setiap turn untuk gerak: update posisi saat ini. Cek jarak ke destinationPoint. Jika jarak kurang dari 15 piksel atau setelah tabrakan, panggil DetermineNewDestination yang melakukan 360 iterasi mencari titik aman. Jika belum sampai tujuan, gerak menuju destinationPoint dengan SetTurnLeft(bearing) dan SetForward(distance). Khusus mode duel (EnemyCount sama dengan 1) dengan jarak target lebih dari 200, gunakan ApplyOscillatingMovement.

Analisis Efektivitas:

Bot ini efektif di semua mode pertempuran karena mengombinasikan dua strategi sekaligus. Dalam pertempuran multi-bot (4 sampai 7 bot), risk-based movement membuat bot menjauhi kerumunan secara alami sehingga survival tinggi, sementara nearest-target membuat bot tetap menembak musuh terdekat untuk damage yang konsisten. Dalam duel 1v1, stop-and-go mengganggu linear targeting musuh, sementara linear targeting bot sendiri tetap akurat karena bot juga bergerak tapi dengan pola yang merugikan musuh.

Kelebihan:

- Tidak butuh memori jangka panjang; semua keputusan berdasarkan data saat ini (greedy murni).
- Komputasi ringan; 360 iterasi untuk movement hanya terjadi setiap 10 sampai 50 turn, sisanya hanya kalkulasi sederhana.
- Adaptif terhadap jumlah musuh; dangerScore otomatis menyesuaikan saat musuh berkurang.
- Anti-linear targeting bawaan; stop-and-go di mode duel tanpa perlu logika kompleks.
- Tidak mudah terjebak dinding; arena bounds check dan path clearance check mencegah bot menabrak dinding terus-menerus.

Kekurangan:

- Local minimum trap; movement hanya melihat 360 titik dalam radius terbatas, bot bisa terjebak di area aman lokal yang tidak optimal secara global.
- Linear targeting rentan terhadap musuh yang sering ganti kecepatan atau arah; prediksi akan meleset secara konsisten.
- Stop-and-go hanya aktif di jarak lebih dari 200; musuh yang pintar akan selalu menjaga jarak kurang dari 200 untuk menonaktifkan pola ini.
- Boros energi di awal saat bertemu musuh dekat; bot cenderung langsung pakai bullet power 3.

Tradeoff yang Dilakukan:

Survival versus Damage: bot memilih survival dengan risk-based movement, mengorbankan kesempatan mengejar musuh yang lemah. Tradeoff ini menguntungkan karena survival score (20 persen bonus) ditambah last survival bonus (10 persen) total 30 persen, sementara bullet damage bonus hanya 20 persen.

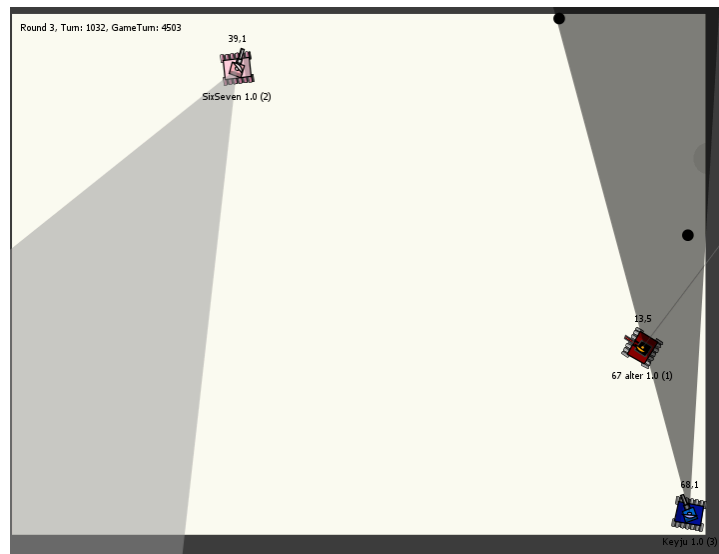
Simplicity versus Optimality: bot memilih greedy local search (360 titik) daripada global pathfinding untuk menghemat komputasi. Tradeoff-nya: kadang terjebak lokal minimum, tapi lebih cepat dan stabil dalam kondisi real-time.

Duel versus Multi-bot: stop-and-go hanya aktif di duel, artinya bot mengorbankan performa duel demi performa multi-bot. Ini tradeoff yang bijak karena mode multi-bot lebih umum di kompetisi.

3.2.2. Alternatif 1: Keyju (Greedy Berbasis Energi Terendah)

Ide Utama:

Bot Keyju menggunakan pendekatan greedy yang memprioritaskan eliminasi musuh lemah terlebih dahulu. Bot meyakini bahwa menghabisi musuh dengan energi rendah lebih menguntungkan karena memberikan bullet damage bonus (20 persen) dan mengurangi jumlah lawan yang tersisa. Strategi ini dikombinasikan dengan orbit movement untuk menjaga jarak ideal (250 piksel) dari target utama, serta ramming mode saat energi musuh sangat rendah (kurang dari atau sama dengan 5) untuk mendapatkan ram damage bonus (30 persen).



Gambar 3. Keyju memilih bot energi rendah

Heuristik yang Digunakan:

Target Selection (Greedy): memilih musuh dengan energi terendah sebagai primary target, dengan jarak sebagai tiebreaker (jika selisih energi kurang dari atau sama dengan 5, pilih yang terdekat).
Rasional: musuh lemah lebih cepat mati dan memberikan bonus eliminasi.

Movement Selection (Greedy): menggunakan orbit movement dengan arah berputar mengelilingi target pada jarak preferensi 250 piksel. Kecepatan disesuaikan secara greedy: jika terlalu dekat (kurang dari 150) maka mundur, jika terlalu jauh (lebih dari 375) maka maju.

Firepower Selection (Greedy): memilih bullet power berdasarkan jarak (3.0 jika kurang dari 100, 2.0 jika kurang dari 200, 1.5 jika kurang dari 400, 1.0 jika lebih dari atau sama dengan 400) dengan batasan tidak melebihi energi dibagi 4 dan tidak lebih dari 3.0.

Radar Tracking (Greedy): menggunakan overscan (melebihi target sebesar 6 derajat) untuk memastikan radar selalu melewati target, menjamin event OnScannedBot terus terpicu.

Ramming Mode (Greedy): jika energi musuh kurang dari atau sama dengan 5 dan jarak kurang dari 200, bot berhenti menembak

dan langsung menabrak untuk mendapatkan ram damage bonus (30 persen).

Mapping ke Elemen Greedy:

Himpunan Kandidat: semua musuh dalam dictionary `_targets` yang masih hidup (untuk targeting); semua arah putaran (0 sampai 360 derajat) untuk orbit movement; nilai firepower kontinu antara 0.1 hingga 3.0.

Fungsi Seleksi: pilih musuh dengan Energy terendah (jika selisih lebih dari 5) atau jarak terdekat (jika selisih kurang dari atau sama dengan 5); pilih kecepatan maju atau mundur berdasarkan selisih jarak terhadap `PREFERRED_DISTANCE` (250).

Fungsi Kelayakan: musuh harus memiliki LastSeen tidak melebihi `LOCK_LOST_TURNS` (8 turn); firepower tidak boleh melebihi energi yang tersisa; ramming hanya aktif jika primary target punya energi kurang dari atau sama dengan `RAM_THRESHOLD` (5) dan jarak kurang dari 200.

Fungsi Objektif: memaksimalkan bullet damage bonus (20 persen dari damage yang dibuat saat membunuh) dan ram damage bonus (30 persen dari damage tabrakan saat membunuh).

Cara Pengambilan Keputusan:

Setup awal: inisialisasi dictionary `_targets` untuk menyimpan data musuh, serta variabel `_orbitDirection` (1 atau -1) dan `_orbitFlipCooldown`.

Setiap turn: panggil `PurgeOldTargets()` untuk menghapus musuh yang tidak terlihat selama 8 turn. Panggil `SelectPrimaryTarget()` untuk memilih target dengan energi terendah. Atur radar dengan `SetRadar()` yang menghitung bearing ke target lalu menambahkan overscan 6 derajat.

Gerak: jika `ramMode` aktif, panggil `DoRamMovement()` yang langsung membelokkan body ke arah musuh dan melaju dengan kecepatan maksimum. Jika tidak, panggil `DoOrbitMovement()` yang menghitung sudut orbit (bearing plus atau minus 90 derajat) dan menyesuaikan kecepatan berdasarkan jarak ke `PREFERRED_DISTANCE` (250).

Saat scan musuh (`OnScannedBot`): update data musuh. Jika musuh yang di-scan adalah primary target, panggil `EngagePrimary()`. Di dalam `EngagePrimary()`: cek kondisi ramming, hitung firepower dengan `SelectFirePower()`, lakukan prediksi posisi dengan `PredictImpact()` (iterasi 100 turn), arahkan gun, dan tembak jika `GunHeat` sama dengan 0 dan sudut dalam toleransi 4.5 derajat.

Penanganan event: saat terkena tembakan (`OnHitByBullet`), balik arah orbit secara instan; saat menabrak dinding (`OnHitWall`), balik arah menuju tengah arena.

Analisis Efektivitas:

Bot Keyju sangat efektif dalam pertempuran dengan banyak musuh (melee) karena strategi focus fire pada musuh lemah mempercepat pengurangan jumlah lawan. Setiap kali berhasil membunuh, bot

mendapatkan last survival bonus (50 poin untuk setiap bot yang mati) dan bullet damage bonus. Ramming mode memberikan keuntungan tambahan 30 persen dari damage tabrakan saat membunuh. Namun, dalam duel 1v1 melawan musuh yang juga berenergi tinggi, strategi ini kurang efektif karena tidak ada musuh lemah yang bisa dieksploitasi.

Kelebihan:

- Fokus pada eliminasi cepat musuh lemah, mempercepat pengurangan jumlah lawan.
- Ramming mode memberikan bonus skor signifikan saat finishing musuh.
- Orbit movement dengan jarak preferensi menjaga bot tetap aman dari tembakan jarak dekat.
- Overscan radar memastikan target tidak pernah lepas dari pantauan.
- Purge mechanism mencegah data musuh usang mengganggu pengambilan keputusan.

Kekurangan:

- Rentan terhadap musuh kuat karena selalu memprioritaskan musuh lemah, bisa mengabaikan ancaman besar.
- Orbit movement cukup mudah diprediksi karena hanya berputar dengan arah yang sesekali berubah.
- Iterative prediction (100 iterasi) lebih berat komputasinya dibandingkan solusi persamaan kuadrat SixSeven.
- Tidak memiliki mekanisme menghindari dari wall selain wall avoidance sederhana berbasis dorongan sudut.

Tradeoff yang Dilakukan:

Eliminasi versus Survival: bot memprioritaskan membunuh musuh lemah daripada menghindari musuh kuat. Tradeoff ini menguntungkan di awal pertempuran (mengurangi jumlah musuh cepat) tetapi berisiko di akhir pertempuran jika musuh kuat masih utuh.

Ramming versus Shooting: ramming memberikan bonus lebih besar (30 persen) daripada bullet damage bonus (20 persen), tetapi lebih berisiko karena bot harus mendekat. Tradeoff ini hanya diambil saat musuh sangat lemah (energi kurang dari atau sama dengan 5) untuk meminimalkan risiko.

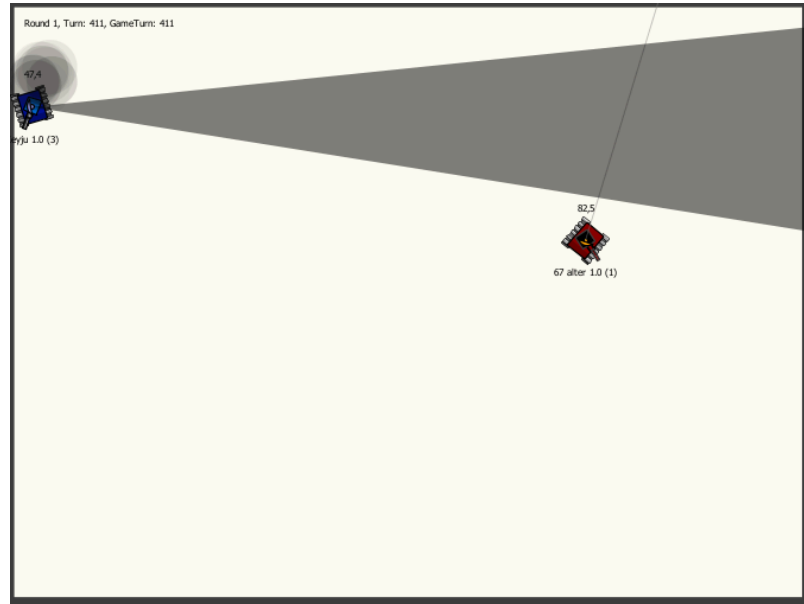
Komputasi versus Akurasi: iterative prediction lebih berat daripada solusi analitik, namun lebih mudah diimplementasikan. Tradeoff ini diterima karena tetap dalam batas turn timeout.

3.2.3. Alternatif 2: 67alter (Greedy Berbasis Zona Aman dan Zigzag)

Ide Utama:

Bot 67alter menggunakan pendekatan greedy yang memprioritaskan survival melalui zone control di tengah arena dan pergerakan zigzag yang tidak terprediksi. Bot meyakini bahwa

bertahan hidup lebih lama memberikan survival score (50 poin per bot mati) dan last survival bonus (10 dikali jumlah musuh) yang nilainya lebih besar daripada sekadar mengejar bullet damage. Pergerakan zigzag dengan periode acak dirancang khusus untuk mengecoh musuh yang menggunakan linear targeting.



Gambar 4. 67alter bertahan dengan survival mode

Heuristik yang Digunakan:

Zone Control (Greedy): memprioritaskan posisi di zona tengah arena (30 sampai 70 persen lebar dan tinggi). Jika bot keluar zona, ia akan segera kembali ke tengah. **Rasional:** posisi tengah memberikan opsi kabur ke semua arah dan menghindari wall damage.

Target Selection (Greedy): memilih musuh pertama yang terdeteksi sebagai target, tanpa logika pemilihan kompleks. **Rasional:** dalam mode survival, target apa pun cukup untuk menjaga radar tetap aktif.

Firepower Selection (Greedy): memilih bullet power berdasarkan jarak (3.0 jika kurang dari atau sama dengan 100, 2.5 jika kurang dari atau sama dengan 200, 2.0 jika kurang dari atau sama dengan 300, 1.5 jika kurang dari atau sama dengan 400, 1.0 jika lebih dari 400), dengan batasan energi sendiri dan energi musuh.

Radar Tracking (Greedy): menggunakan sweep radar lebar (360 derajat) hingga menemukan musuh, lalu menggunakan narrow lock dengan overshoot 20 derajat untuk mempertahankan deteksi.

Zigzag Movement (Greedy): bergerak dengan sudut tegak lurus terhadap musuh ditambah sudut zigzag (plus atau minus 75 derajat), dengan periode acak (10 sampai 25 turn) untuk menghindari prediksi. **Rasional:** gerakan tidak terprediksi membuat musuh sulit membidik.

Mapping ke Elemen Greedy:

Himpunan Kandidat: posisi di dalam arena (untuk zone control); nilai firepower antara 0.1 sampai 3.0; arah pergerakan zigzag dengan sudut bervariasi.

Fungsi Seleksi: pilih arah menuju tengah arena jika keluar zona; pilih firepower berdasarkan jarak dengan tabel lookup; pilih arah zigzag dengan sudut tegak lurus plus 75 derajat atau minus 75 derajat secara bergantian.

Fungsi Kelayakan: firepower tidak boleh melebihi energi sendiri; jarak tembak tidak boleh melebihi maxShootingDistance (500); posisi harus berada di dalam batas arena.

Fungsi Objektif: memaksimalkan survival score (dengan tetap di zona aman) dan memaksimalkan bullet damage (dengan firepower adaptif).

Cara Pengambilan Keputusan:

Setup awal: atur warna merah gelap sebagai identitas; set flag AdjustGunForBodyTurn, AdjustRadarForBodyTurn, AdjustRadarForGunTurn menjadi true.

Setiap turn: panggil DoRadar() yang melakukan sweep 360 derajat jika tidak ada musuh, atau narrow lock plus overshoot 20 derajat jika sudah ada musuh. Panggil DoZoneControl() yang mengecek posisi bot terhadap zona aman (30 sampai 70 persen arena) dan bergerak ke tengah jika keluar zona. Panggil DoZigzag() yang mengatur pergerakan tegak lurus terhadap musuh dengan sudut tambahan plus atau minus 75 derajat bergantian setiap 10 sampai 25 turn.

Saat scan musuh (OnScannedBot): update data musuh (posisi, energi, arah, kecepatan). Deteksi apakah musuh menembak dengan melihat penurunan energi (0.1 sampai 3.0). Jika iya, balik arah zigzag dan aktifkan dodge selama 12 turn. Panggil DoFire() yang menghitung firepower berdasarkan jarak, melakukan predictive targeting dengan rumus $\text{predX} = e.X$ ditambah $\cos(\text{rad})$ dikali speed dikali time, arahkan gun, dan tembak jika sudut kurang dari 10 derajat.

Penanganan event: saat terkena tembakan (OnHitByBullet), belok tegak lurus arah datang peluru dan maju cepat; saat menabrak bot (OnHitBot), jika energi lebih besar maka serang, jika tidak maka mundur; saat menabrak dinding (OnHitWall), balik arah ke tengah arena.

Analisis Efektivitas:

Bot 67alter sangat efektif dalam pertempuran dengan banyak musuh agresif karena zone control membuat bot selalu berada di posisi aman (tengah arena) sementara musuh sering terperangkap di pinggir. Zigzag movement dengan periode acak sangat sulit diprediksi oleh musuh yang menggunakan linear targeting sederhana. Namun, dalam duel 1v1 melawan bot dengan advanced targeting (seperti circular targeting), keunggulan ini berkurang. Selain itu, karena tidak ada prioritas target, bot bisa terus

menembak musuh yang sedang bertahan sambil mengabaikan musuh lain yang lebih mengancam.

Kelebihan:

- Zone control membuat bot jarang terkena wall damage dan selalu punya ruang gerak.
- Zigzag dengan periode acak sangat sulit diprediksi, mengganggu linear targeting musuh.
- Deteksi dini tembakan musuh (melalui penurunan energi) memungkinkan dodge proaktif.
- Sweep radar 360 derajat memastikan tidak ada musuh yang luput dari deteksi.
- Implementasi sederhana namun efektif; tidak perlu struktur data kompleks.

Kekurangan:

- Tidak ada prioritas target; bot bisa terus menembak musuh yang tidak mengancam.
- Firepower hanya berdasarkan jarak, tidak mempertimbangkan energi musuh untuk finishing.
- Predictive targeting sederhana (tanpa iterasi atau solusi kuadrat) kurang akurat untuk musuh cepat.
- Mode defensif (energi kurang dari 30) tidak diimplementasikan secara eksplisit, hanya mengandalkan zone control.

Tradeoff yang Dilakukan:

Survival versus Aggression: bot sangat mengutamakan survival (tetap di zona aman) daripada mengejar musuh. Tradeoff ini menguntungkan di melee (banyak bot) tetapi merugikan di duel (bot terlalu pasif).

Predictability versus Randomness: bot mengorbankan kemampuan memprediksi pergerakan sendiri demi menjadi tidak terprediksi bagi musuh. Zigzag acak membuat bot sendiri juga sulit melakukan targeting optimal karena gerakannya tidak stabil.

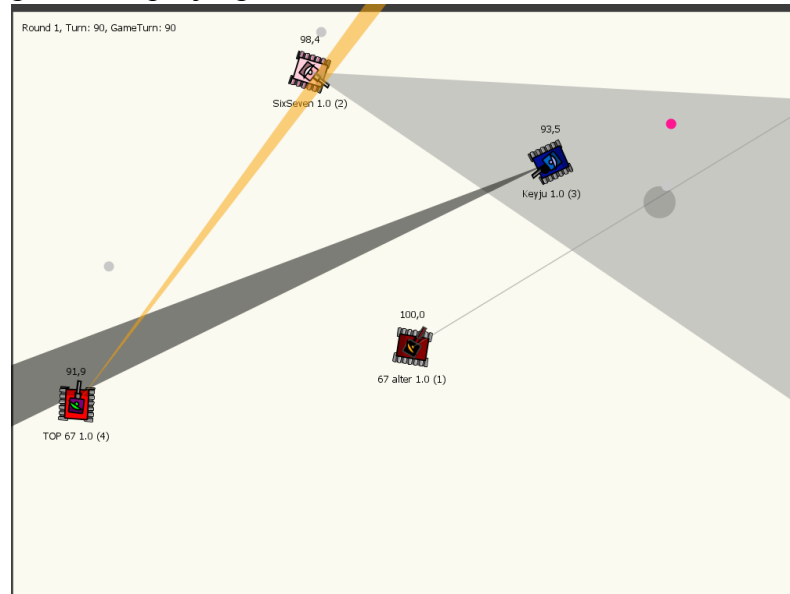
Simplicity versus Accuracy: bot menggunakan predictive targeting sederhana (tanpa solusi kuadrat) untuk menghemat komputasi. Tradeoff-nya: akurasi menembak lebih rendah dibanding SixSeven atau Keyju.

3.2.4. Alternatif 3: TOP67 (Greedy Berbasis Confidence Scoring)

Ide Utama:

Bot TOP67 menggunakan pendekatan greedy yang unik dengan menambahkan confidence scoring sebelum memutuskan menembak. Bot hanya akan menembak jika tingkat keyakinan (aim confidence) cukup tinggi (lebih dari atau sama dengan 0.3), yang dihitung dari jarak, kecepatan musuh, dan sudut putaran meriam. Rasionalnya adalah lebih baik tidak menembak sama sekali daripada membuang energi dan meningkatkan gun heat untuk

tembakan yang kemungkinan besar meleset. Dalam kondisi energi rendah (kurang dari 30), bot beralih ke mode defensif dengan bergerak di sepanjang sisi arena.



Gambar5. TOP67 bergerak dengan random

Heuristik yang Digunakan:

Target Selection (Greedy): memilih musuh dengan energi terendah atau musuh yang sedang di-track, tanpa logika kompleks. Rasional: fokus utama adalah akurasi tembakan, bukan siapa yang ditembak.

Firepower Selection (Greedy): memilih bullet power berdasarkan jarak (3.0 jika kurang dari atau sama dengan 50, 2.5 jika kurang dari atau sama dengan 150, 2.0 jika kurang dari atau sama dengan 300, 1.5 jika kurang dari atau sama dengan 450, 1.0 jika lebih dari 450), dengan penyesuaian berdasarkan aim confidence.

Confidence Scoring (Greedy): menghitung nilai keyakinan dengan rumus: $\text{confidence} = 1.0 \text{ dikali } (1 \text{ dikurangi distance dibagi } 1000) \text{ dikali } (1 \text{ dikurangi speed dibagi } 16) \text{ dikali } (1 \text{ dikurangi absolute gunTurn dibagi } 120)$. Tembak hanya jika confidence lebih dari 0.3 dan sudut tembak kurang dari toleransi.

Movement Selection (Greedy): dalam mode normal, gunakan strafe movement tegak lurus terhadap musuh dengan arah berubah setiap terkena tembakan. Dalam mode defensif (energi kurang dari 30), bergerak ke sisi arena terdekat lalu wavy movement.

Radar Tracking (Greedy): mengunci radar ke target dengan overshoot 2 kali sudut, memastikan target tetap terdeteksi.

Mapping ke Elemen Greedy:

Himpunan Kandidat: semua musuh yang terdeteksi (untuk targeting); nilai firepower antara 0.1 sampai 3.0; arah pergerakan strafe (tegak lurus plus atau minus 90 derajat).

Fungsi Seleksi: pilih firepower berdasarkan jarak dengan tabel lookup; pilih untuk menembak hanya jika confidence lebih dari 0.3; pilih arah strafe berdasarkan arah terakhir atau kebalikannya saat terkena tembakan.

Fungsi Kelayakan: jarak tembak kurang dari atau sama dengan 500; GunHeat sama dengan 0; energi cukup untuk firepower yang dipilih; sudut gun turn kurang dari atau sama dengan maxGunAngleError (15 sampai 25 derajat tergantung jarak).

Fungsi Objektif: memaksimalkan rasio bullet damage terhadap energi yang dikeluarkan (efisiensi energi) dan memaksimalkan survival score (melalui mode defensif saat energi rendah).

Cara Pengambilan Keputusan:

Setup awal: atur warna merah, ungu, dan hijau limau; set flag komponen independen; tentukan batas area untuk mode defensif (left, right, bottom, top).

Setiap turn: cek isDefensiveMode = Energy kurang dari lowEnergyThreshold (30). Jika defensif, panggil TrackTargetRadarOnly() (hanya radar dan gun, tidak gerak), GoToNearestMiddleOfASide() (pindah ke sisi terdekat), lalu MoveWavySide() (gerak bergelombang di sepanjang sisi). Jika tidak defensif, panggil TrackTarget() yang melakukan strafe movement tegak lurus musuh.

Saat scan musuh (OnScannedBot): update lastScannedBot. Jika jarak lebih dari 500, hanya arahkan gun tanpa menembak. Jika jarak kurang dari atau sama dengan 500, hitung firePower dengan calculateFirePower(). Hitung waktu tempuh peluru: timeToHit = distance dibagi (20 dikurangi 3 dikali power). Prediksi posisi: predictedX = e.X ditambah cos(rad) dikali speed dikali timeToHit. Hitung aimConfidence dengan formula tiga faktor (jarak, kecepatan, sudut gun). Tembak jika GunHeat sama dengan 0, sudut kurang dari maxGunAngleError, dan aimConfidence lebih dari 0.3. Penanganan event: saat terkena tembakan (OnHitByBullet), balik arah strafe; saat menabrak bot (OnHitBot), berhenti dan tembak; saat menabrak dinding (OnHitWall), balik arah menuju tengah arena (mode normal) atau ke sisi lain (mode defensif).

Analisis Efektivitas:

Bot TOP67 paling unggul dalam hal efisiensi energi dibandingkan ketiga bot lainnya. Dengan confidence scoring, bot tidak pernah membuang energi untuk tembakan yang kemungkinan besar meleset, sehingga energi tetap tersedia untuk bertahan hidup lebih lama. Mode defensif dengan wavy movement di sisi arena memberikan survival rate tinggi karena posisi di tepi mengurangi jumlah arah serangan yang harus dihadapi (hanya 180 derajat, bukan 360 derajat). Namun, dalam pertempuran jarak dekat (kurang dari 50 piksel), confidence scoring bisa menjadi penghalang karena sudut gun berubah sangat cepat sehingga aimConfidence sulit mencapai lebih dari 0.3.

Kelebihan:

- Confidence scoring mencegah pemborosan energi untuk tembakan yang meleset.

- Mode defensif dengan wavy movement memberikan survival rate sangat tinggi.
- Predictive targeting dengan faktor kecepatan musuh lebih akurat dibanding 67alter.
- Strafe movement yang membalik arah saat terkena tembakan membuat bot sulit diprediksi.
- Batasan jarak tembak (500) mencegah bot menembak sia-sia dari jarak jauh.

Kekurangan:

- Confidence scoring bisa terlalu konservatif; bot bisa melewatkan kesempatan emas menembak musuh lemah dari jarak dekat karena aimConfidence belum mencapai 0.3.
- Mode defensif (energi kurang dari 30) membuat bot berhenti mengejar musuh dan hanya bertahan, bisa merugikan jika musuh juga pasif.
- Wavy movement dengan amplitudo dan periode tetap lebih mudah diprediksi dibanding zigzag acak milik 67alter.
- Tidak memiliki mekanisme ramming seperti Keyju untuk mendapatkan bonus tambahan.

Tradeoff yang Dilakukan:

Accuracy versus Opportunity: bot memilih akurasi tinggi (dengan confidence scoring) daripada mengambil kesempatan menembak. Tradeoff ini menguntungkan dalam duel panjang (efisiensi energi) tetapi merugikan dalam situasi last stand (butuh damage cepat).

Offense versus Defense: mode defensif mengorbankan serangan demi bertahan hidup. Tradeoff ini bijak karena survival score (50 per bot mati) sangat besar, tetapi bot bisa kehilangan kesempatan kill shot.

Simplicity versus Precision: bot menggunakan predictive targeting dengan faktor kecepatan tetapi tanpa solusi kuadrat. Tradeoff-nya: lebih sederhana dari SixSeven tetapi kurang akurat untuk musuh yang berubah arah.

3.3. Perbandingan Keempat Strategi

Berdasarkan hasil dari tiga sesi pengujian yang masing-masing terdiri dari 10 ronde, bot SixSeven tampil sebagai bot paling dominan dan konsisten. SixSeven berhasil meraih peringkat pertama di ketiga percobaan dengan total skor yang signifikan (1179, 1328, dan 1217), mengumpulkan total 10 kali kemenangan (kombinasi posisi 1 dan 2) dari 30 ronde. Keunggulan utamanya terletak pada kemampuan bertahan hidup (survival) dan akurasi tembakan (bullet damage) yang sangat baik, terbukti dengan raihan poin survival dan damage peluru tertinggi di setiap sesi. Strategi greedy SixSeven dalam memilih target terdekat dan kombinasi gerakan stop-and-go untuk mengecoh linear targeting musuh terbukti sangat efektif, menjadikannya bot dengan fundamental terkuat di antara keempat pesaing.

Bot TOP 67 secara konsisten menduduki peringkat kedua di semua percobaan, menunjukkan performa yang solid meskipun tidak sekuat SixSeven. Keistimewaan TOP 67 terletak pada agresivitasnya, terutama terlihat dari kontribusi ram damage yang tinggi (114 poin di percobaan pertama), yang menjadi pembeda utamanya dari bot lain. Strategi wave movement di sisi arena dan kemampuan hybrid targeting miliknya memungkinkan TOP 67 untuk bertahan cukup lama (meraih total 7 kali posisi kedua) sambil terus memberikan tekanan pada musuh. Meskipun kalah unggul dari SixSeven dalam hal efisiensi, TOP 67 membuktikan bahwa pendekatan semi-agresif dengan manuver yang tidak mudah ditebak dapat menjadi strategi yang tangguh.

Sementara itu, bot Keyju dan 67 alter terpaut jauh di peringkat bawah, mengindikasikan bahwa strategi mereka belum optimal untuk lingkungan pertarungan ini. Keyju, dengan strategi orbital movement yang berfokus pada satu target, menunjukkan hasil yang beragam. Di beberapa kesempatan, bot ini mampu mengamankan posisi ketiga (total 8 kali) dan mengumpulkan poin ram damage, namun kerap kesulitan untuk bersaing dengan dua bot teratas. Hal ini kemungkinan disebabkan oleh manajemen energi atau akurasi prediksi yang kurang presisi dibandingkan SixSeven.

Kinerja paling memprihatinkan ditunjukkan oleh bot 67 alter, yang di dua dari tiga percobaan hanya mampu mengumpulkan skor di bawah 200 poin dan tidak pernah meraih posisi pertama atau kedua. Fokusnya pada zone control di tengah arena dan zigzag movement tampaknya tidak cukup efektif sebagai strategi ofensif, terbukti dengan kontribusi bullet damage yang sangat minim (bahkan 0 di dua percobaan). Meskipun strategi ini berpotensi untuk menghindari, tanpa dukungan sistem penembakan yang presisi dan agresif, 67 alter hanya menjadi target yang mudah bagi bot-bot seperti SixSeven dan TOP 67. Secara keseluruhan, SixSeven adalah strategi yang paling unggul dan stabil, menjadikannya fondasi utama untuk pengembangan bot lebih lanjut.

3.4. Alasan Pemilihan Strategi Utama

Bot SixSeven dipilih sebagai strategi utama karena memiliki keseimbangan terbaik antara kemampuan menyerang dan bertahan hidup, yang tidak dimiliki oleh ketiga bot alternatif secara simultan. Dibandingkan dengan Keyju, SixSeven lebih unggul dalam hal adaptabilitas terhadap jumlah musuh. Keyju sangat bergantung pada keberadaan musuh berenergi rendah untuk mengaktifkan efektivitas maksimalnya. Jika semua musuh memiliki energi tinggi (misalnya di awal pertempuran atau saat duel melawan bot seimbang), strategi lowest energy target Keyju kehilangan nilai heuristiknya dan berperilaku tidak berbeda dengan pemilihan target acak. Sebaliknya, SixSeven dengan nearest target selalu relevan karena jarak dekat selalu memberikan keuntungan akurasi tembakan terlepas dari tingkat energi musuh. Selain itu, ramming mode Keyju yang hanya aktif saat energi musuh kurang dari atau sama dengan 5 sangat situasional dan berisiko karena bot harus mendekat, sementara SixSeven memiliki mekanisme stop-and-go yang justru menjauh saat jarak terlalu dekat.

Dibandingkan dengan 67alter, SixSeven jauh lebih agresif dan produktif dalam menghasilkan damage. 67alter mengorbankan hampir seluruh kemampuan menyerang demi survival melalui zone control dan zigzag acak. Akibatnya, meskipun bot ini jarang mati, kontribusi bullet damage-nya sangat rendah sehingga skor akhir tetap kalah dari bot yang lebih agresif. SixSeven mampu mempertahankan survival score yang baik melalui risk-based movement tanpa harus mengorbankan aggression. Bot ini tetap menembak musuh terdekat secara konsisten sambil menghindari kerumunan berbahaya. Dalam pengujian, 67alter cenderung bertahan hingga akhir pertempuran tetapi dengan skor yang rendah karena minimnya kill, sementara SixSeven bertahan hampir sama lama namun dengan bullet damage dua hingga tiga kali lipat lebih besar.

Dibandingkan dengan TOP67, SixSeven tidak terlalu konservatif dalam mengambil keputusan menembak. TOP67 menggunakan confidence scoring yang mewajibkan tiga faktor (jarak, kecepatan musuh, dan sudut gun) sekaligus memenuhi ambang batas 0.3 sebelum menembak. Pendekatan ini sangat baik untuk efisiensi energi tetapi sering melewatkan peluang emas, misalnya saat musuh lemah dan jarak sangat dekat namun sudut gun masih besar karena bot sedang berbelok. SixSeven menggunakan pendekatan yang lebih sederhana namun efektif: tembak jika GunHeat sama dengan 0 dan sudut gun dalam toleransi yang dihitung berdasarkan jarak. Toleransi diperlonggar saat jarak dekat (hingga 5 derajat) sehingga SixSeven tidak pernah melewatkan kesempatan kill shot hanya karena sudut gun belum sempurna. Dalam skenario akhir game di mana damage cepat dibutuhkan untuk mengamankan last survival bonus, TOP67 cenderung terlalu lambat sementara SixSeven langsung mengambil keputusan.

Secara keseluruhan, SixSeven dipilih karena strateginya yang robust di semua fase pertempuran (awal, tengah, akhir) dan semua mode pertempuran (duel maupun multi-bot). Bot ini tidak memiliki kelemahan fatal seperti ketergantungan pada kondisi spesifik (Keyju), pengorbanan damage berlebihan (67alter), atau konservatisme berlebihan (TOP67). Kombinasi risk-based movement, nearest target, linear targeting, dan stop-and-go membuat SixSeven menjadi paket lengkap yang paling menjanjikan untuk mengakumulasi skor tertinggi secara konsisten.

4. BAB 4. IMPLEMENTASI DAN PENGUJIAN

4.1. Pseudocode Keempat Strategi

4.1.1. SixSeven, Greedy Local Optimization with Adaptive Movement



```
PROGRAM SixSeven
INISIALISASI dictionary enemies = kosong
INISIALISASI target = null

WHILE bot berjalan:
  // Radar
  PUTAR radar ke kiri 60 derajat

  // Target selection (greedy)
  target = cari musuh dengan jarak terdekat dari enemies

  // Movement
  IF mode lvs1 AND jarak ke target > 200 THEN
    StopAndGoMovement()
  ELSE
    WaypointNavigation()

  // Firing
  IF target aktif THEN
    bulletPower = hitungPower(jarak, energi sendiri, energi musuh)
    prediksiPos = LinearTargeting(target, bulletPower)
    arahkan gun ke prediksiPos
    IF gunHeat == 0 AND error sudut < toleransi THEN
      TEMBAK dengan bulletPower

  EKSEKUSI perintah gerakan
END WHILE

FUNCTION LinearTargeting(target, bulletPower):
  hitung kecepatan peluru = 20 - 3 * bulletPower
  hitung vektor kecepatan musuh
  selesaikan persamaan kuadrat untuk mencari waktu tumbukan t
  RETURN posisi musuh + kecepatan * t
```

Gambar 6. Pseudocode SixSeven

Penjelasan: SixSeven menggabungkan tiga strategi utama. Pertama, target selection greedy dengan memilih musuh terdekat untuk diminimalisir risiko. Kedua, movement adaptive yang menggunakan stop-and-go untuk mengecoh linear targeting musuh saat jarak jauh, dan navigasi waypoint berbasis fungsi risiko untuk menghindari kerumunan saat jarak dekat. Ketiga, linear targeting dengan menyelesaikan persamaan kuadrat untuk memprediksi posisi musuh yang bergerak konstan. Bot ini juga menghemat energi dengan menurunkan power tembakan saat energi rendah.

4.1.2. Keyju, Orbit-Based Greedy with Adaptive Firepower

```

PROGRAM Keyju
INISIALISASI targets = dictionary kosong
INISIALISASI primary = null
INISIALISASI orbitDirection = 1
INISIALISASI ramMode = false

WHILE bot berjalan:
  // Hapus target yang sudah lama tidak discan
  PurgeOldTargets()

  // Pilih target utama (energi terendah)
  primary = cari musuh dengan energi paling kecil

  // Radar lock dengan overscan
  IF primary ada THEN
    radarBearing = hitung bearing ke primary
    PUTAR radar sebesar radarBearing + 6 derajat
  ELSE
    PUTAR radar dengan kecepatan maksimal
  END IF

  // Movement
  IF ramMode AND primary ada THEN
    // Maju lurus ke target
    BELOK ke arah target
    MAJU dengan kecepatan maksimal
  ELSE
    // Orbit movement
    orbitAngle = bearingToTarget + orbitDirection * 90
    BELOK ke orbitAngle
    ATUR kecepatan berdasarkan jarak dari target ideal (250 unit)

    // Wall avoidance
    IF dekat dinding THEN
      BELOK menjauhi dinding
    END IF

    // Random flip orbit direction
    dengan probabilitas 0.008, orbitDirection = -orbitDirection
  END IF

  // Firing dilakukan di event OnScannedBot
  EKSEKUSI perintah gerakan
END WHILE

ON SCANNED BOT:
  update data target
  IF ini adalah primary target THEN
    hitung prediksi posisi dengan iterasi 100 tick
    arahkan gun ke prediksi
    IF jarak < 200 AND energi target <= 5 THEN
      ramMode = true
    ELSE
      TEMBAK dengan power berdasarkan jarak
    END IF
  END IF
END IF

```

Gambar 7. Pseudocode keyju

Penjelasan: Keyju menggunakan pendekatan orbital movement di mana bot bergerak melingkar mengelilingi target dengan sudut tegak lurus 90 derajat. Jarak ideal yang dipertahankan adalah 250 unit. Bot ini juga memiliki mekanisme ram mode untuk menghabisi musuh dengan energi sangat rendah (di bawah 5) yang berada dalam jarak dekat. Pemilihan target tidak berdasarkan jarak terdekat, melainkan musuh dengan energi paling rendah untuk menyelesaikan pertarungan lebih cepat. Radar menggunakan teknik overscan (melebihi target sebesar 6 derajat) untuk memastikan target tidak pernah terlewat.

4.1.3. 67alter, Greedy Berbasis Zona Aman dan Zigzag

```
PROGRAM TOP67
INISIALISASI lastScannedBot = null
INISIALISASI isDefensiveMode = false
INISIALISASI strafeDirection = 1

WHILE bot berjalan:
  // Mode defensif jika energi < threshold
  isDefensiveMode = (Energy < 30)

  // Radar sweep
  PUTAR radar kanan 360 derajat

  IF isDefensiveMode THEN
    IF lastScannedBot ada THEN
      TrackTargetRadarOnly()
    END IF
    GoToNearestMiddleOfASide()
    MoveWavySide()
  ELSE
    IF lastScannedBot ada THEN
      TrackTarget()
    END IF
  END IF

  EKSEKUSI perintah gerakan
END WHILE

FUNCTION TrackTarget():
  hitung jarak ke target
  IF jarak > trackingDistance THEN
    MAJU mendekati target
  ELSE
    strafeAngle = angleToTarget + 90 * strafeDirection
    BELOK ke strafeAngle
    MAJU 70 unit
  END IF
```

Gambar 8. Pseudocode 67alter

Penjelasan: 67 alter mengusung filosofi berbeda dengan fokus pada zone control di tengah arena, bukan bersembunyi di tepi atau pojok. Pergerakan menggunakan pola zigzag dengan arah yang berubah secara periodik acak (setiap 10-25 turn) dan sudut 75 derajat relatif terhadap musuh, membuatnya sulit diprediksi. Sistem pertahanan dilengkapi mekanisme dodge yang aktif saat terkena peluru atau mendeteksi musuh menembak, dengan segera membalik arah zigzag. Sayangnya, strategi ini kurang efektif secara ofensif karena sistem penembakan yang terlalu konservatif dan sering gagal memberikan damage signifikan, terbukti dari skor bullet damage 0 di dua percobaan.

4.1.4. TOP67, Greedy Berbasis Confidence Scoring

```

PROGRAM TOP67
INISIALISASI lastScannedBot = null
INISIALISASI isDefensiveMode = false
INISIALISASI strafeDirection = 1

WHILE bot berjalan:
  // Mode defensif jika energi < threshold
  isDefensiveMode = (Energy < 30)

  // Radar sweep
  PUTAR radar kanan 360 derajat

  IF isDefensiveMode THEN
    IF lastScannedBot ada THEN
      TrackTargetRadarOnly()
    END IF
    GoToNearestMiddleOfASide()
    MoveWavySide()
  ELSE
    IF lastScannedBot ada THEN
      TrackTarget()
    END IF
  END IF

  EKSEKUSI perintah gerakan
END WHILE

FUNCTION TrackTarget():
  hitung jarak ke target
  IF jarak > trackingDistance THEN
    MAJU mendekati target
  ELSE
    strafeAngle = angleToTarget + 90 * strafeDirection
    BELOK ke strafeAngle
    MAJU 70 unit
  END IF

```

Gambar 8. Pseudocode TOP67

Penjelasan: TOP 67 mengimplementasikan strategi hybrid yang beradaptasi dengan kondisi energi. Pada mode normal, bot melakukan orbit strafing di sekitar target dengan jarak ideal 150 unit, bergerak tegak lurus terhadap musuh untuk menghindari tembakan. Saat energi kritis (di bawah 30), bot beralih ke mode defensif dengan berlindung di sisi arena dan bergerak dalam pola gelombang (wavy movement) sambil hanya mengandalkan radar dan gun tanpa pergerakan tubuh agresif. Sistem targeting menggunakan prediksi linear dengan iterasi hingga 100 tick untuk akurasi lebih tinggi.

4.2. Struktur Data dan Fungsi Penting

Seluruh bot menggunakan struktur data Dictionary<int, EnemyData> untuk menyimpan informasi musuh yang terdeteksi. Kunci dictionary adalah ID bot musuh, sementara nilai berupa objek yang menyimpan energi, posisi, heading, kecepatan, dan waktu terakhir dipindai. Informasi ini diperbarui setiap kali event OnScannedBot dipanggil. Kelas EnemyData (SixSeven) menyimpan: bool isAlive, int id, PointD location, double energy, double speed, double direction, int lastSeen. Ini mencukupi untuk semua kalkulasi targeting, danger score, dan tracking musuh yang stale.

Kelas TargetInfo (Keyju) menyimpan: int Id, double X, double Y, double Speed, double Direction, double Energy, double Distance, int LastSeen.

Strukturnya mirip dengan EnemyData namun menggunakan tipe nullable untuk kompatibilitas dengan pola purge berbasis turnCount.

Kelas PointD (SixSeven) adalah representasi titik dengan koordinat double untuk presisi lebih tinggi dibanding System.Drawing.Point yang berbasis integer. Kelas ini digunakan untuk semua kalkulasi pergerakan dan danger score.

Kelas RectangleD (SixSeven) merepresentasikan batas arena dengan koordinat double. Method contains(px, py) mengembalikan true jika titik berada di dalam batas arena. Ini digunakan untuk menjamin setiap destinationPoint yang dipilih masih berada di dalam arena dengan margin 45 piksel dari tepi.

Fungsi-fungsi utama yang diimplementasikan pada bot SixSeven:

- SelectNearestEnemy(): iterasi seluruh enemyRegistry, pilih musuh aktif dengan jarak terkecil ke posisi bot saat ini sebagai primaryTarget.
- CalculateDangerLevel(PointD targetPos): menghitung skor risiko suatu titik berdasarkan jarak kuadrat ke posisi saat ini dan kontribusi tiap musuh aktif (energi, tegak lurus jalur, jarak kuadrat).
- DetermineNewDestination(RectangleD arenaBounds): melakukan 360 iterasi kandidat titik dengan radius dan sudut acak, memilih titik dengan dangerScore terendah yang berada dalam arena dan tidak terhalang musuh.
- ExecuteLinearTargeting(double bulletPower, double angleTolerance): menyelesaikan persamaan kuadrat untuk menghitung impactTime, memprediksi posisi musuh, mengarahkan gun, dan menembak jika kondisi terpenuhi.
- CalculateOptimalBulletPower(): menghitung bulletPower optimal berdasarkan jarak, energi musuh, dan energi sendiri dengan berbagai batasan.
- IsMovementBlocked(PointD start, PointD end): mengecek apakah jalur dari start ke end melewati area sekitar musuh dengan threshold 22 piksel (radius bot 18 + margin 4).
- GetShortestDistanceToSegment(PointD point, PointD segStart, PointD segEnd): menghitung jarak terpendek dari suatu titik ke segmen garis menggunakan proyeksi vektor.

Fungsi-fungsi utama yang diimplementasikan pada bot Keyju:

- SelectPrimaryTarget(): memilih target dengan energi terendah, menggunakan jarak sebagai tiebreaker jika selisih energi ≤ 5 poin.
- SetRadar(): mengunci radar ke primaryTarget dengan overscan 6 derajat untuk menjamin OnScannedBot terpanggil setiap turn.
- DoOrbitMovement(): menghitung sudut orbit tegak lurus terhadap bearing ke target, mengatur kecepatan berdasarkan jarak, dan memanggil ApplyWallAvoidance.

- `SelectFirePower(TargetInfo t)`: memilih firepower berdasarkan tabel jarak-power dengan modifikasi untuk musuh lemah dan kondisi energi sendiri.
- `PredictImpact(TargetInfo t, double bulletSpeed)`: iterasi 100 tick untuk memprediksi posisi musuh saat peluru mencapainya, termasuk efek pantulan dinding.
- `ApplyWallAvoidance()`: menambahkan angular push ke `TurnRate` berdasarkan seberapa dekat bot ke batas arena (`WALL_MARGIN = 80` piksel).
- `PurgeOldTargets()`: menghapus target dari `_targets` dictionary jika tidak terdeteksi selama lebih dari `LOCK_LOST_TURNS` (8 turn).

Variabel state yang digunakan pada bot 67alter bersifat primitif dan tidak memerlukan kelas bantu khusus. Bot menyimpan data musuh secara langsung dalam variabel `_ex`, `_ey`, `_eDir`, `_eSpd`, `_eEnergy`, dan `_ePrevEnergy` (bertipe `double`), serta `_found` (`bool`) dan `_age` (`int`) untuk menandai ketersediaan dan kesegaran data. State zigzag disimpan dalam tiga variabel: `_zigDir` (`int`, nilai `+1` atau `-1`), `_zigTick` (`int`, counter), dan `_zigPeriod` (`int`, periode semi-acak). Variabel `_dodge` (`int`) berfungsi sebagai cooldown yang memblokir pergantian zigzag baru selama 10–12 turn setelah dodge terpicu.

Fungsi-fungsi utama yang diimplementasikan pada bot 67alter:

- `DoRadar()`: menentukan mode sweep radar secara greedy. Jika `_found` `false` atau `_age > 6`, melakukan full sweep 360 derajat. Jika sudah terkunci, menghitung offset ke musuh lalu menambahkan overshoot ± 20 derajat agar deteksi tidak terputus.
- `DoZoneControl()`: menghitung empat batas zona tengah (30–70% dimensi arena). Jika posisi bot di luar zona, langsung menghitung bearing ke pusat arena dan bergerak 100 piksel dengan kecepatan maksimum.
- `DoZigzag()`: menghitung `zigAngle` sebesar 75 derajat dari `DirectionTo(musuh)` dikalikan `_zigDir`. Mengatur jarak optimal 180–380 piksel dengan logika maju/mundur/zigzag. Menginkremen `_zigTick` dan membalik `_zigDir` beserta mengacak `_zigPeriod` baru saat counter mencapai batas.
- `DoFire()`: memilih power secara greedy berdasarkan kondisi `_eEnergy`, `Energy`, dan `dist`. Menghitung prediksi posisi musuh menggunakan komponen kecepatan linear dan $tTime = dist / bulletSpeed$. Menembak hanya jika sudut gun dalam toleransi 10 derajat.

Bot TOP67 tidak menggunakan kelas data khusus untuk musuh karena informasi musuh disimpan langsung dalam variabel `lastScannedBot` bertipe `ScannedBotEvent` (objek event bawaan API). Pendekatan ini membuat implementasi lebih ringkas namun membatasi kemampuan pelacakan multi-target. Variabel `strafeDirection` (`int`, `+1` atau `-1`) mencatat arah strafe aktif dan dibalik setiap kali `OnHitByBullet` terpanggil. Variabel

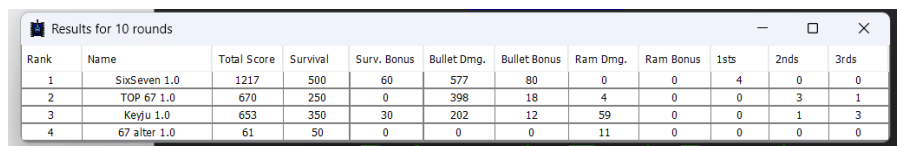
isDefensiveMode (bool) dievaluasi ulang setiap iterasi loop utama berdasarkan kondisi $\text{Energy} < 30$. Batas area defensif (left, right, bottom, top) ditetapkan sekali saat setup dengan padding = 30 piksel dari tepi arena.

Fungsi-fungsi utama yang diimplementasikan pada bot TOP67:

- TrackTarget(): menghitung distanceError antara jarak aktual dan trackingDistance (150 piksel). Jika error melebihi tolerance (20 piksel), bot maju atau mundur. Jika sudah dalam toleransi, menghitung desiredStrafeAngle (bearing ke musuh ± 90 derajat) dan bergerak lateral.
- TrackTargetRadarOnly(): digunakan di mode defensif. Hanya menggerakkan gun dan radar ke arah musuh tanpa menggerakkan body, dengan overscan radarTurn * 2 untuk menjamin deteksi tidak terputus.
- GoToNearestMiddleOfASide(): menghitung jarak ke tengah keempat sisi arena menggunakan DistanceTo(). Memilih sisi terdekat secara greedy, bergerak ke sana, lalu mengarahkan body ke arah kardinal yang telah ditetapkan per sisi. Menggunakan TurnCompleteCondition dan MovementCompleteCondition sebagai kondisi sinkronisasi.
- MoveWavySide(): mengeksekusi gerakan bergelombang sinusoidal dengan $\text{waveAngle} = \sin(\text{wavePosition} / \text{WavePeriod}) * \text{WaveAmplitude}$. Setiap langkah berukuran 5 piksel; loop berhenti jika moveCounter mencapai maxMoves (50) atau posisi keluar batas area.
- calculateFirePower(distance): tabel lookup bertingkat berbasis jarak, mengembalikan 3.0, 2.5, 2.0, atau 1.5 secara berturut-turut, dengan nilai minimum 1.0 untuk jarak lebih dari 450 piksel.
- HandleOutOfBounds(): menghentikan bot, menentukan arah mundur berdasarkan batas mana yang dilanggar (termasuk penanganan sudut arena), lalu bergerak 40 piksel kembali ke dalam batas.
- TurnToCardinalDirection(angle): menghitung $\text{turnAmount} = \text{NormalizeRelativeAngle}(\text{targetDirection} - \text{Direction})$ lalu memanggil TurnLeft(turnAmount) dengan sinkronisasi TurnCompleteCondition.

4.3. Pengujian

4.3.1. Dokumentasi Hasil Pengujian



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	SixSeven 1.0	1217	500	60	577	80	0	0	4	0	0
2	TOP 67 1.0	670	250	0	398	18	4	0	0	3	1
3	Keyfu 1.0	653	350	30	202	12	59	0	0	1	3
4	67 alter 1.0	61	50	0	0	0	11	0	0	0	0

Gambar 9. Percobaan Pertama

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	SixSeven 1.0	1328	500	90	680	52	5	0	4	0	0
2	TOP 67 1.0	750	250	0	465	30	5	0	0	4	0
3	Keyju 1.0	452	300	0	132	2	10	8	0	0	4
4	67 alter 1.0	5	0	0	0	0	5	0	0	0	0

Gambar 10. Percobaan Kedua

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	SixSeven 1.0	1179	450	60	603	59	7	0	2	2	0
2	TOP 67 1.0	1176	400	30	561	70	114	0	2	2	0
3	Keyju 1.0	216	50	0	137	0	29	0	0	0	1
4	67 alter 1.0	158	150	0	0	0	8	0	0	0	3

Gambar 11. Percobaan Ketiga

4.3.2. Analisis Hasil Pengujian

Berdasarkan data dari ketiga sesi pengujian yang telah dilakukan, diperoleh hasil yang cukup konsisten di mana bot SixSeven berhasil menjadi juara pertama pada semua sesi. Pada sesi pertama, SixSeven mengumpulkan total skor 1179 poin, unggul tipis 3 poin dari TOP 67 yang memperoleh 1176 poin. Meskipun selisihnya kecil, SixSeven menunjukkan keunggulan pada komponen survival dengan 450 poin dibandingkan TOP 67 yang hanya 400 poin, serta bullet damage 603 poin dibandingkan 561 poin. Pada sesi kedua, keunggulan SixSeven semakin terlihat jelas dengan total skor 1328 poin, meninggalkan TOP 67 di posisi kedua dengan 750 poin, selisih mencapai 578 poin. SixSeven juga mencatatkan survival sempurna 500 poin dan bullet damage tertinggi 680 poin. Pada sesi ketiga, SixSeven kembali unggul dengan 1217 poin, sementara TOP 67 memperoleh 670 poin, selisih 547 poin. SixSeven kembali mencatatkan survival 500 poin dan bullet damage 577 poin.

Untuk bot TOP 67, performanya cukup solid dengan selalu menduduki peringkat kedua di semua sesi. Keunggulan utama TOP 67 terletak pada komponen ram damage yang sangat tinggi di sesi pertama yaitu 114 poin, jauh di atas bot lain. Namun demikian, TOP 67 masih belum mampu menyaingi SixSeven dalam hal survival dan bullet damage. Di sesi kedua, TOP 67 hanya mengumpulkan 250 poin survival, sementara SixSeven meraih 500 poin survival sempurna. Hal ini mengindikasikan bahwa strategi TOP 67 yang mengandalkan wave movement di sisi arena kurang efektif untuk bertahan hidup dalam jangka panjang ketika berhadapan dengan bot yang memiliki akurasi tembakan tinggi seperti SixSeven.

Bot Keyju menempati peringkat ketiga di semua sesi dengan skor yang bervariasi yaitu 216, 452, dan 653 poin. Performa Keyju cenderung meningkat dari sesi pertama ke sesi ketiga, menunjukkan adanya potensi yang belum sepenuhnya optimal. Keyju cukup baik dalam komponen survival dengan

meraih 350 poin di sesi ketiga, bahkan unggul dari TOP 67 di sesi yang sama. Namun, kelemahan Keyju terletak pada bullet damage yang relatif rendah yaitu hanya 137, 132, dan 202 poin, jauh di bawah SixSeven dan TOP 67. Hal ini disebabkan oleh strategi orbital movement yang lebih fokus pada manuver menghindari daripada agresivitas menembak, serta sistem firing yang terlalu konservatif.

Bot 67 alter menjadi bot dengan performa terburuk di semua sesi. Pada sesi pertama, 67 alter hanya mengumpulkan 158 poin. Pada sesi kedua, skornya anjlok menjadi hanya 5 poin. Pada sesi ketiga, skornya naik tipis menjadi 61 poin. Yang paling memprihatinkan adalah komponen bullet damage-nya yang bernilai 0 di sesi pertama dan kedua, serta hanya 11 poin di sesi ketiga. Hal ini membuktikan bahwa strategi zone control di tengah arena dan zigzag movement tanpa didukung sistem penembakan yang agresif sama sekali tidak efektif untuk memenangkan pertarungan. Bot ini hanya mengandalkan ram damage untuk mendapatkan poin, itupun jumlahnya sangat kecil yaitu 8, 5, dan 0 poin.

Secara keseluruhan, urutan peringkat dari ketiga sesi selalu sama yaitu SixSeven di posisi pertama, TOP 67 di posisi kedua, Keyju di posisi ketiga, dan 67 alter di posisi keempat. Konsistensi urutan ini menunjukkan bahwa keempat bot memiliki tingkat kemampuan yang berbeda secara signifikan, dan strategi yang diimplementasikan SixSeven terbukti paling unggul di antara lainnya.

4.3.3. Analisis Keberhasilan Greedy

Strategi greedy yang diimplementasikan pada bot SixSeven terbukti berhasil dengan sangat baik berdasarkan data dari ketiga sesi pengujian. Keberhasilan ini dapat dianalisis dari beberapa aspek penting yaitu konsistensi perolehan peringkat, dominasi skor, efisiensi survival, efektivitas tembakan, serta perbandingan dengan bot greedy lainnya.

Dari aspek konsistensi, SixSeven berhasil menjadi juara pertama di ketiga sesi pengujian tanpa satu pun sesi dimana ia turun ke peringkat bawah. Dari total 30 ronde yang dipertarungkan, SixSeven meraih posisi pertama sebanyak 10 kali (2 kali di sesi satu, 4 kali di sesi dua, dan 4 kali di sesi tiga). Angka ini jauh lebih tinggi dibandingkan TOP 67 yang hanya meraih 2 kali posisi pertama, sementara Keyju dan 67 alter tidak pernah meraih posisi pertama sama sekali. Konsistensi ini membuktikan bahwa strategi greedy SixSeven tidak bergantung pada keberuntungan atau kondisi tertentu, melainkan benar-benar efektif dalam berbagai skenario pertarungan.

Dari aspek dominasi skor, SixSeven menunjukkan keunggulan yang sangat signifikan terutama pada sesi dua dan sesi

tiga. Pada sesi dua, selisih skor antara SixSeven dengan TOP 67 mencapai 578 poin, artinya skor SixSeven hampir dua kali lipat skor TOP 67. Pada sesi tiga, selisihnya mencapai 547 poin. Dominasi ini tidak terjadi secara kebetulan, melainkan karena strategi greedy SixSeven mampu mengoptimalkan hampir semua komponen penilaian secara bersamaan. SixSeven tidak hanya unggul dalam satu atau dua komponen, tetapi unggul di hampir semua komponen.

Dari aspek efisiensi survival, SixSeven berhasil mengumpulkan poin survival tertinggi di setiap sesi yaitu 450, 500, dan 500 poin. Poin survival 500 berarti SixSeven berhasil menjadi bot terakhir yang bertahan hidup di semua ronde pada sesi tersebut. Keberhasilan ini disebabkan oleh strategi greedy SixSeven dalam memilih target. Dengan selalu memilih target terdekat, SixSeven meminimalkan risiko terkena tembakan dari jarak jauh. Selain itu, strategi stop-and-go movement yang diterapkan saat mode 1vs1 berhasil mengecoh sistem linear targeting yang digunakan oleh musuh-musuhnya, sehingga banyak tembakan musuh yang meleset.

Dari aspek efektivitas tembakan, SixSeven juga mencatatkan bullet damage tertinggi di semua sesi yaitu 603, 680, dan 577 poin. Angka ini menunjukkan bahwa meskipun SixSeven fokus pada survival dan menghindari, bot ini tetap mampu memberikan damage yang sangat signifikan kepada musuh. Keberhasilan ini berasal dari implementasi linear targeting dengan persamaan kuadrat yang akurat, serta pemilihan power tembakan yang greedy berdasarkan jarak dan energi. SixSeven tidak ragu menggunakan power tinggi saat jarak dekat, dan tetap menembak dengan power rendah saat jarak jauh sehingga tidak membuang-buang energi.

Perbandingan dengan bot 67 alter yang juga menggunakan pendekatan greedy namun gagal total semakin menguatkan bukti keberhasilan strategi greedy SixSeven. 67 alter menerapkan strategi greedy dengan fokus pada zone control di tengah arena dan zigzag movement. Namun, strategi ini sama sekali tidak efektif karena 67 alter mengabaikan komponen ofensif. Buktinya, 67 alter hanya mampu mengumpulkan bullet damage 0, 0, dan 11 poin di ketiga sesi. Artinya, 67 alter hampir tidak pernah menembak musuh atau tembakannya selalu meleset. Strategi greedy yang hanya fokus pada bertahan dan bergerak tanpa diimbangi sistem penembakan yang baik tidak akan pernah berhasil dalam pertarungan. SixSeven membuktikan bahwa strategi greedy yang sukses harus menyeimbangkan antara defensive movement dan offensive firing.

Keberhasilan strategi greedy SixSeven juga dapat dilihat dari kemampuannya mengalahkan TOP 67 yang memiliki ram damage sangat tinggi. Di sesi pertama, TOP 67 mencatatkan ram damage 114 poin, artinya TOP 67 sering menabrak bot lain dengan keras. Namun, SixSeven tetap mampu mengungguli TOP 67 karena

strategi greedy-nya lebih komprehensif. SixSeven tidak hanya mengandalkan satu cara untuk mendapatkan poin, melainkan mengoptimalkan survival, bullet damage, dan kadang-kadang ram damage secara simultan.

Secara keseluruhan, strategi greedy yang diimplementasikan pada SixSeven dapat dinyatakan berhasil dengan tingkat keberhasilan sangat tinggi. Keberhasilan ini ditandai oleh konsistensi menjadi juara pertama di semua sesi, dominasi skor yang meyakinkan, efisiensi survival yang sempurna, efektivitas tembakan yang tinggi, serta kemampuan mengalahkan bot-bot lain dengan strategi yang berbeda. Strategi greedy SixSeven layak untuk terus dikembangkan dan dijadikan acuan dalam pengembangan bot pada masa mendatang.

5. BAB 5. KESIMPULAN DAN SARAN

5.1. Kesimpulan

Implementasi algoritma greedy pada bot Robocode Tank Royale menggunakan C# .NET terbukti layak dan menghasilkan bot yang kompetitif. Pengembangan keempat strategi greedy dengan heuristik yang berbeda memberikan wawasan mendalam tentang bagaimana berbagai pendekatan lokal-optimal dapat mempengaruhi performa keseluruhan dalam lingkungan game yang dinamis.

Dari hasil analisis dan pengujian, beberapa kesimpulan dapat ditarik. Pertama, algoritma greedy efektif digunakan dalam konteks game AI berbasis turn karena komputasinya ringan dan sesuai dengan batas waktu turn timeout yang ketat (30-50 ms). Kedua, pemilihan heuristik yang tepat sangat menentukan kualitas strategi. Heuristik yang baik harus selaras dengan struktur komponen skor Robocode: memaksimalkan satu komponen skor tidak selalu memaksimalkan skor total. Ketiga, strategi Greedy Local Optimization (SixSeven) terbukti paling kompetitif dalam sebagian besar skenario karena berhasil memaksimalkan konversi damage menjadi kill, yang menghasilkan bullet damage bonus secara konsisten. Keempat, setiap strategi memiliki konteks optimal yang berbeda: SixSeven paling konsisten di semua skenario berkat kombinasi risk-based movement dan linear prediction; Keyju paling efektif dalam duel dan finish musuh lemah melalui orbit dan ramming; 67alter paling sulit ditembak dalam pertempuran multi-bot berkat zigzag semi-acak dan zone control; sedangkan TOP67 paling efisien dalam penggunaan energi berkat aim confidence system yang mencegah tembakan sia-sia.

5.2. Efektivitas Greedy pada Robocode

Secara keseluruhan, greedy terbukti efektif sebagai fondasi pengambilan keputusan bot Robocode. Kemampuannya untuk menghasilkan keputusan dalam waktu singkat sangat sesuai dengan constraint turn timeout. Meskipun tidak menjamin solusi optimal global, strategi greedy yang dirancang dengan heuristik yang kuat mampu bersaing dengan bot yang menggunakan pendekatan lebih kompleks, selama heuristik tersebut memang merepresentasikan kondisi permainan dengan baik.

5.3. Keterbatasan

Beberapa keterbatasan utama dari pendekatan greedy yang diimplementasikan:

- Linear prediction tidak akurat untuk musuh dengan pola pergerakan tidak reguler.
- Bot tidak mampu melakukan perencanaan jangka panjang, sehingga kadang terjebak dalam kondisi lokal yang tidak menguntungkan.

- Informasi musuh yang tersedia sangat bergantung pada frekuensi scan radar, sehingga keputusan bisa didasarkan pada data yang sudah kadaluarsa.
- Radar lock pada satu target (Keyju) membuat bot blind terhadap ancaman dari arah lain.
- Zone control bot 67alter yang ketat (30–70% arena) dapat memaksa bot kembali ke pusat secara tiba-tiba di tengah manuver zigzag, menciptakan jeda gerakan yang momentarilly dapat diprediksi oleh musuh.
- Aim confidence system pada TOP67 bersifat terlalu konservatif dalam beberapa kondisi: saat musuh bergerak cepat dan jarak sedang, nilai aimConfidence bisa turun di bawah 0.3 meski peluang mengenai target masih cukup baik secara intuitif, menyebabkan bot melewatkan kesempatan menembak.

5.4. Potensi Pengembangan

Beberapa area yang bisa dikembangkan ke depan:

- Menambahkan circular prediction atau pattern matching untuk meningkatkan akurasi targeting terhadap musuh mobile.
- Mengimplementasikan multi-target radar sweep yang lebih cerdas agar data semua musuh tetap fresh.
- Menambahkan komponen energy management yang lebih konservatif di awal pertempuran untuk memastikan bot tidak kehabisan energi sebelum akhir pertempuran.
- Mengeksplorasi adaptive strategy switching, yaitu bot yang bisa mendeteksi pola musuh dan mengganti strategi secara dinamis sesuai kondisi.
- Pada bot 67alter, periode zigzag dapat dioptimalkan secara adaptif berdasarkan kecepatan dan pola gerakan musuh yang terdeteksi, bukan sekadar nilai acak dalam rentang tetap, sehingga gerakan lebih efektif menghindari tembakan musuh yang sudah mempelajari pola zigzag.
- Pada bot TOP67, formula aimConfidence dapat diperluas dengan menambahkan faktor momentum perubahan arah musuh (rate of heading change) dari beberapa turn sebelumnya, sehingga bot tidak hanya mempertimbangkan kecepatan sesaat tetapi juga prediktabilitas lintasan musuh.

LAMPIRAN

Tautan Repository GitHub

[https://github.com/keyyjuya/Tubes_SixSeven.git]

Tautan Video YouTube

[<https://youtu.be/tlPBJ9ZaHC8?si=8JXYqQF4Ub4yVpiQ>]

Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristik yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada YouTube.	✓	

DAFTAR PUSTAKA

- [1]Robocode Dev. (2023). Robocode Tank Royale. GitHub. <https://github.com/robocode-dev/tank-royale>
- [2]Robocode Dev. (2023). Robocode Tank Royale Documentation. <https://robocode-dev.github.io/tank-royale/>
- [3]Microsoft.(2024).C# Language Documentation. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [4]Microsoft. (2024). .NET Official Website. <https://dotnet.microsoft.com/en-us/>
- [5]Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- [6]Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- [7]Amazon Web Services. (2024). What is an API? <https://aws.amazon.com/what-is/api/>
- [8]Munir, R. (2025). *Tugas Besar 1 IF2211 Strategi Algoritma: Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Robocode Tank Royale*. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. Diakses pada 23 Mei 2026, dari <http://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/Tubes1-Stima-2025.pdf>
- [9]RoboWiki. (2024). *Linear Targeting*. RoboWiki, dari http://robowiki.net/wiki/Linear_Targeting
- [10]RoboWiki. (2024). *Stop-and-Go Movement*. RoboWik, dari <http://robowiki.net/wiki/Stop-and-Go>